BEGINNER BUILD · 11 STEPS

Plain LLM vs RAG in LangGraph

PLAIN LLM

Guesses. Knows nothing about your docs.

VS

RAG APP

Retrieves your text, then answers from it.



Chandra Sekhar
AI & Tech

→ Swipe

THE IDEA

What is RAG?

00

Retrieval-Augmented Generation means the model answers using **your documents**, not just its training data.

1. **Retrieve: find the most relevant text chunks**
2. **Generate: answer using only those chunks**
3. **Return the answer grounded in your data**

Why it matters

Ask about your company handbook and get a real answer instead of a confident guess.

Two nodes. One graph. A working chatbot in under 15 minutes.



Chandra Sekhar
AI & Tech

Follow me

STEP 01

Project structure

01

Start clean. Keep documents apart from code, and give each file exactly one job.

layout

```
rag-langgraph/
  docs/           # your knowledge
  index.py        # build the vector DB
  rag_graph.py    # the LangGraph app
  run.py          # CLI chat loop
```

RULE OF THUMB

- One file, one responsibility
- Docs never mixed with code
- Index once, query many times

Structure first saves debugging later.



Chandra Sekhar
AI & Tech

Follow me

STEP 02

Install the packages

02

terminal

```
pip install -U langgraph langchain \
  langchain-community chromadb tiktoken \
  langchain-text-splitters
pip install -U langchain-openai
```

LangGraph
Orchestrates the RAG workflow.

Chroma
Vector database for embeddings.

LangChain
Loaders and text splitters.

Four installs and you have the whole stack.



Chandra Sekhar
AI & Tech

Follow me

STEP 03

Add your documents

03

Drop **.txt** or **.md** files into **/docs**. Each file becomes searchable knowledge.

```
docs/

faq.txt
handbook.txt
product_notes.md
```

Few docs

Thin, repetitive answers.

MORE DOCS

Richer, more complete answers.

Start small

Try 3 to 5 files you know well, so you can judge whether the answers are correct.

> **Your data is the product. The code is just plumbing.**



Chandra Sekhar
AI & Tech

Follow me

STEP 04

Load documents

04

Read every file in **/docs** and convert it into LangChain Document objects.

index.py

```
from pathlib import Path
from langchain_community.document_loaders import TextLoader

DOCS_DIR = Path(&#x27;docs')

def load_docs():
    docs = []
    for fp in DOCS_DIR.glob(&#x27;*'):
        if fp.is_file():
            docs.extend(TextLoader(str(fp),
                encoding=&#x27;utf-8').load())
    return docs
```

Ready for chunking.



Chandra Sekhar
AI & Tech

Follow me

STEP 05

Split into chunks

05

Big documents search badly. Break them into small pieces: **800 chars** per chunk with **120 overlap**.

index.py

```
from langchain_text_splitters import (
    RecursiveCharacterTextSplitter)

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=120,
```

1. Load files from /docs
2. Split into 800-char chunks
3. Embed and persist in Chroma

> **Overlap keeps context from getting cut at chunk boundaries.**



Chandra Sekhar
AI & Tech

Follow me

STEP 06

Embed and store

06

Convert chunks into vectors and persist them in Chroma. Run it **once**.

index.py

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

Chroma.from_documents(
    documents=chunks,
    embedding=OpenAIEmbeddings(),
    persist_directory=&#x27;chroma_db',
    collection_name=&#x27;rag_docs',
)
print(&#x27;Index built')
```

Run it

export OPENAI_API_KEY=... then python index.py

Similarity search is now instant.



Chandra Sekhar
AI & Tech

Follow me

STEP 07

Define the graph state

07

State is the data that flows through your graph. A TypedDict keeps it typed and predictable.

rag_graph.py

```
from typing import TypedDict, List
from langchain_core.documents import Document

class RAGState(TypedDict):
    question: str
    docs: List[Document]
    answer: str
```

THREE FIELDS

- question: the input
- docs: what retrieval found
- answer: the final output

Every node reads and writes this state.



Chandra Sekhar
AI & Tech

Follow me

STEP 08

Build the retrieve node

08

First node: search the vector DB for the **top 4** matching chunks and put them in state.

rag_graph.py

```
vectorstore = Chroma(
    persist_directory=&#x27;chroma_db',
    collection_name=&#x27;rag_docs',
    embedding_function=OpenAIEmbeddings(),
)
retriever = vectorstore.as_retriever(
    search_kwargs={&#x27;k': 4})

def retrieve(state: RAGState) → RAGState:
    q = state[&#x27;question']
    docs = retriever.invoke(q)
    return {&#x27;question': q, 'docs': docs, 'answer': ''}
```

Retrieval done.



Chandra Sekhar
AI & Tech

Follow me

STEP 09

Set up the LLM and context

Initialize **gpt-4o-mini**, pull the question and docs from state, and merge the chunks into one context string.

rag_graph.py

```
llm = ChatOpenAI(model=&#x27;gpt-4o-mini',
                 temperature=0)

def generate(state: RAGState) → RAGState:
    q = state[&#x27;question']
    docs = state[&#x27;docs']
    # combine chunks into one context
    context = &#x27;\n\n'.join(
        [d.page_content for d in docs])
```

Why k=4

Enough context to answer well, small enough to keep the prompt cheap and focused.

temperature=0 keeps answers consistent and grounded.

Chandra Sekhar
AI & Tech

Follow me

STEP 10

Finish the generate node

10

The system message is your guardrail: **answer only from the context**.

rag_graph.py

```
    messages = [
        SystemMessage(content=(
            &#x27;Answer using ONLY the context. '
            "If missing, say you don&#x27;t know.")),
        HumanMessage(content=
            f&#x27;Question: {q}\n\nContext:\n{context}')
    ]
```

WHAT THE LLM SEES

- The user question
- Only the retrieved chunks
- A strict system rule

Tip

Print the context while testing to see exactly what the model was given.

That one prompt line reduces hallucination.



Chandra Sekhar
AI & Tech

Follow me

STEP 11

Wire the graph

11

Define the flow and let LangGraph run it: **START, retrieve, generate, END**.

rag_graph.py

```
def build_graph():
    g = StateGraph(RAGState)
    g.add_node(&#x27;retrieve', retrieve)
    g.add_node(&#x27;generate', generate)

    g.add_edge(START, &#x27;retrieve')
    g.add_edge(&#x27;retrieve', 'generate')
    g.add_edge(&#x27;generate', END)
    return g.compile()

rag_app = build_graph()
```

Adding nodes later is one more add_edge.



Chandra Sekhar
AI & Tech

Follow me

STEP 12

Run your RAG app

12

A simple CLI loop: ask a question, get an answer, see how many chunks were used.

run.py

```
from rag_graph import rag_app

while True:
    q = input(&#x27;Ask: ').strip()
    if not q:
        break
    r = rag_app.invoke(
        {&#x27;question': q, 'docs': [], 'answer': ''})
    print(r[&#x27;answer'])
    print(&#x27;Chunks used:', len(r['docs']))
```

Run

python run.py

Your RAG chatbot is live.



Chandra Sekhar
AI & Tech

Follow me

NEXT STEPS

Make it production-ready

13

Source citations
Show which files were used.

Relevance filtering
Drop low-score chunks.

Fallback handling
Handle the no-match case.

Better retrieval
Reranking, multi-query, hybrid search.

citations

```
sources = [d.metadata for d in r[&#x27;docs']]
```

Each upgrade is just another node in the graph.



Chandra Sekhar
AI & Tech

Follow me

YOUR TURN

Docs in.
Grounded answers out.

You now have a full RAG pipeline in three small files.
Build it, then extend it.

Save Repost Comment

Follow for more AI builds